DISTRIBUTED BANKING TRANSACTION SIMULATOR

KUALA LUMPUR UNIVERSITY OF SCIENCE AND TECHNOLOGY
(formerly known as Infrastructure University Kuala Lumpur, IUKL)



Distributed and Parallel Computing · Group Project

Lecturer: AP.TS.Dr Abudhahir

Syl Group

- Hussein Musab Abdi
- Yasin Suleyman Abdulkadir
- Mohamed Mohamed Hussein
- Abdirizak Abdirahman Abdikarim
- Mohamed Luqman Hassan

PROBLEM AND OBJECTIVES

- Many customers change the same accounts at the same time
- No coordination → money lost or created **(race condition)**
- Opposite transfers → both wait forever **(deadlock)**
- **Goal:** serve many customers quickly and keep every balance correct

What we built

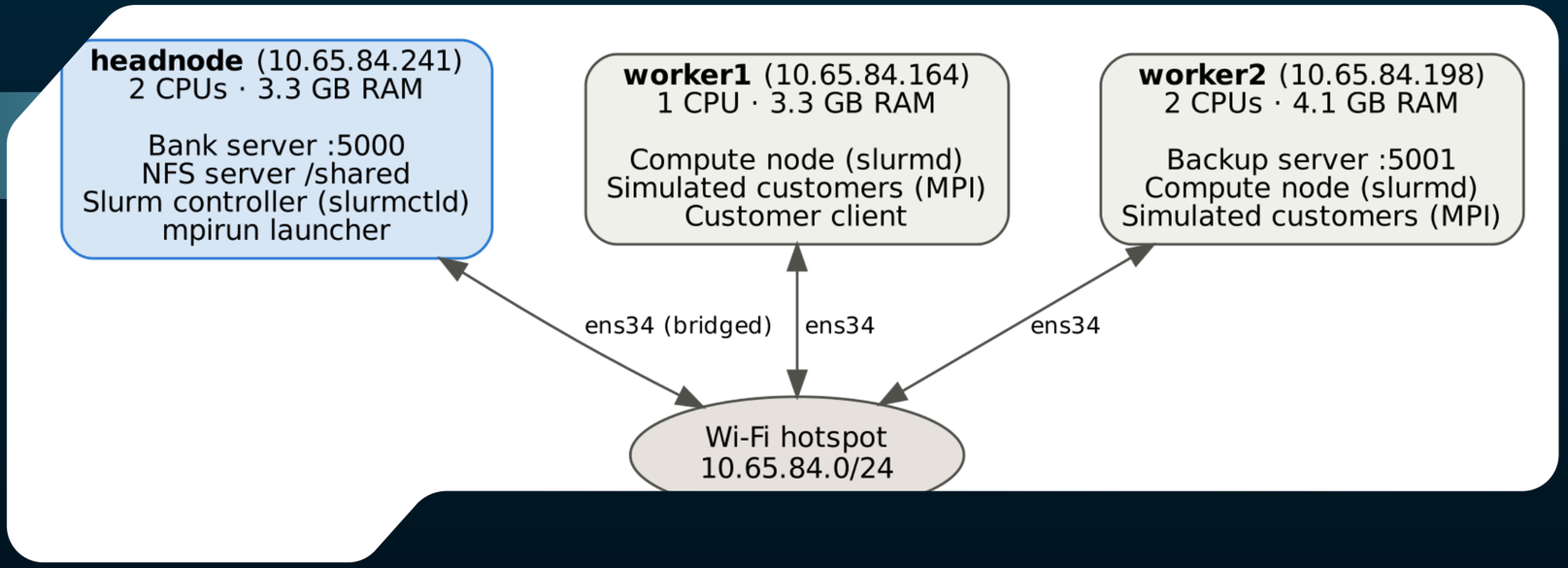
Bank server, 8 worker threads
None / global / per-account locks
Lock accounts in numerical order
Backup server, writer and replicator threads
Load simulator with mpirun on a Slurm cluster

Module concept

Sockets, client–server, thread pool
Synchronization, mutex
Deadlock prevention
Replication, producer–consumer
MPI, cluster computing



OUR CLUSTER

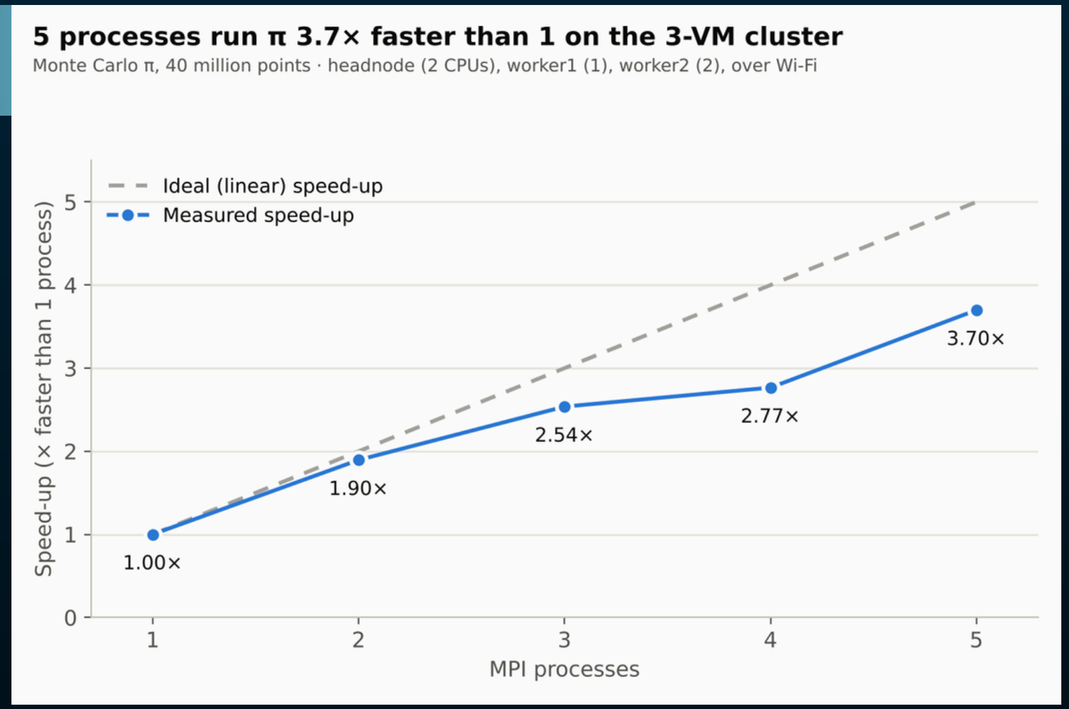


- 3 laptops, each running one Ubuntu VM in VMware (lab PCs had no internet)
- **headnode:** bank server, shared folder, Slurm controller
- **worker1, worker2:** compute nodes; worker2 also runs the backup
- Wi-Fi hotspot · passwordless SSH · NFS /shared · Open MPI · Slurm

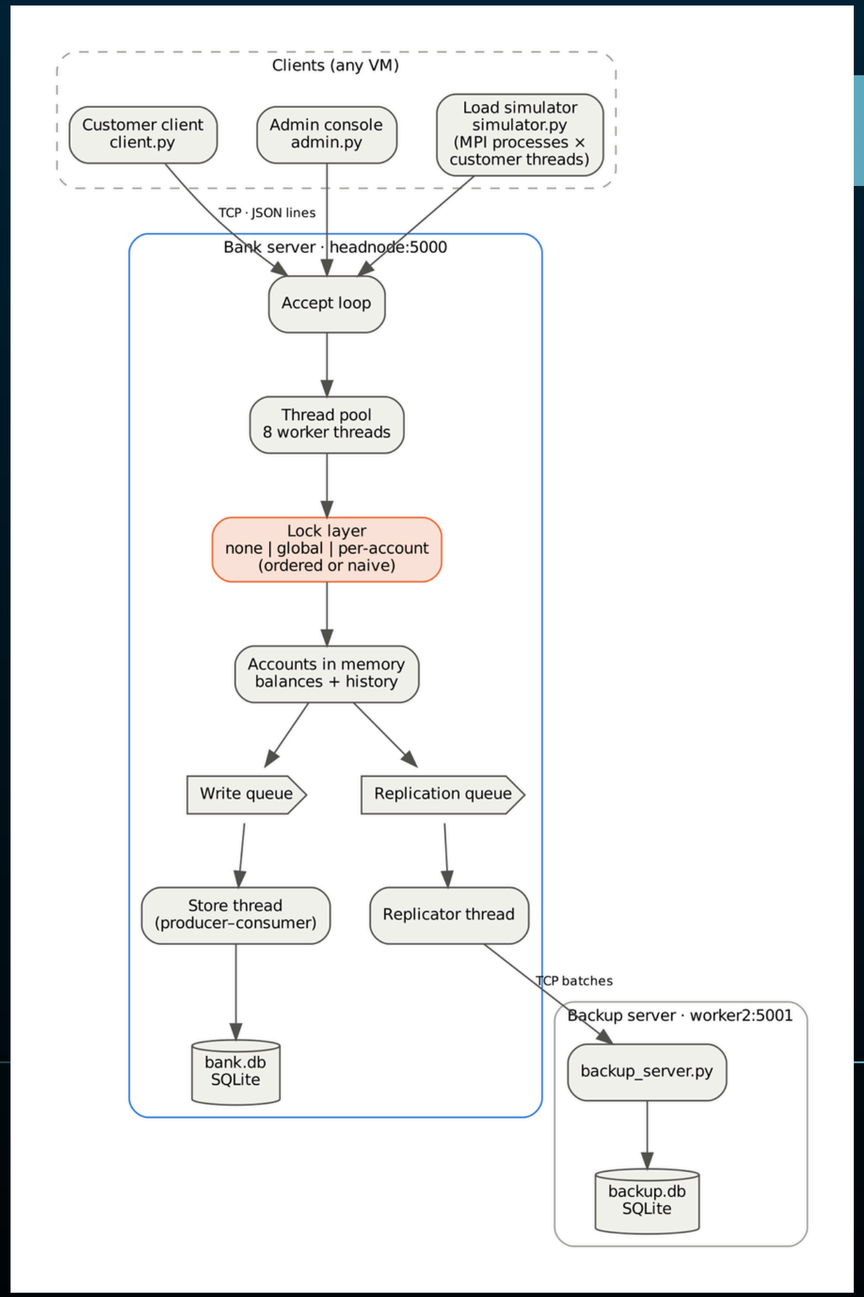
```
mpiuser@headnode:/shared$ mpirun --hostfile hostfile -np 6 --mca btl_tcp_if_include ens34 --mca btl_tcp_disable_family 6 python3 hello.py
Rank 0 of 6 running on headnode
Rank 5 of 6 running on worker2
Rank 3 of 6 running on worker1
Rank 2 of 6 running on worker1
Rank 1 of 6 running on headnode
Rank 4 of 6 running on worker2
mpiuser@headnode:/shared$ mpirun --hostfile hostfile -np 6 --mca btl_tcp_if_include ens34 --mca btl_tcp_disable_family 6 python3 pi.py
pi = 3.141530  processes = 6  time = 1.44 s
mpiuser@headnode:/shared$
```

SLURM AND PARALLEL SPEED-UP

- **Slurm:** 3 nodes idle, batch job computed π on 5 processes
- 5 MPI processes ran 3.7× faster than 1 (6.14 s → 1.66 s)
- **Wi-Fi:** about 15 Mbit/s and 22 ms delay, much slower than a wired network



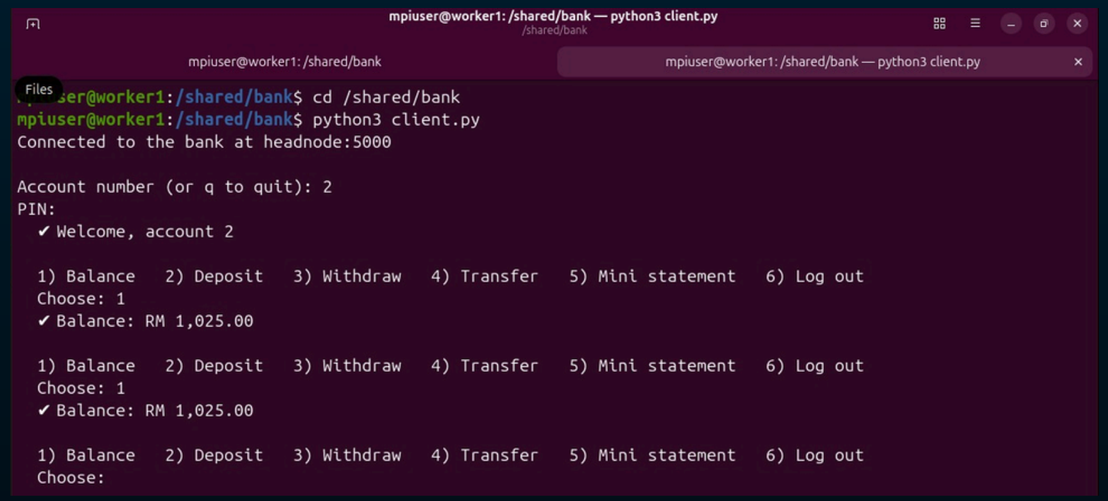

```
mpiuser@headnode:/shared$ sudo systemctl enable --now slurmd
Synchronizing state of slurmd.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable slurmd
mpiuser@headnode:/shared$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
main*        up   infinite      3   idle headnode,worker[1-2]
mpiuser@headnode:/shared$ srun
srun: fatal: No command given to execute.
mpiuser@headnode:/shared$ srun -N3 hostname
headnode
worker2
worker1
mpiuser@headnode:/shared$ cat > /shared/pi_job.sh << 'EOF'
#!/bin/bash
#SBATCH --job-name=pi
#SBATCH --nodes=3
#SBATCH --ntasks=5
#SBATCH --output=/shared/pi_%j.out
CDROM mca btl_tcp_if_include ens34 --mca btl_tcp_disable_family 6 python3 /shared/pi.py 40000000
EOF
sbatch /shared/pi_job.sh
squeue
Submitted batch job 2
             JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
                 2      main       pi  mpiuser PD       0:00      3 (None)
mpiuser@headnode:/shared$ cat /shared/pi_*.out
pi = 3.141276  processes = 5  time = 1.53 s
```



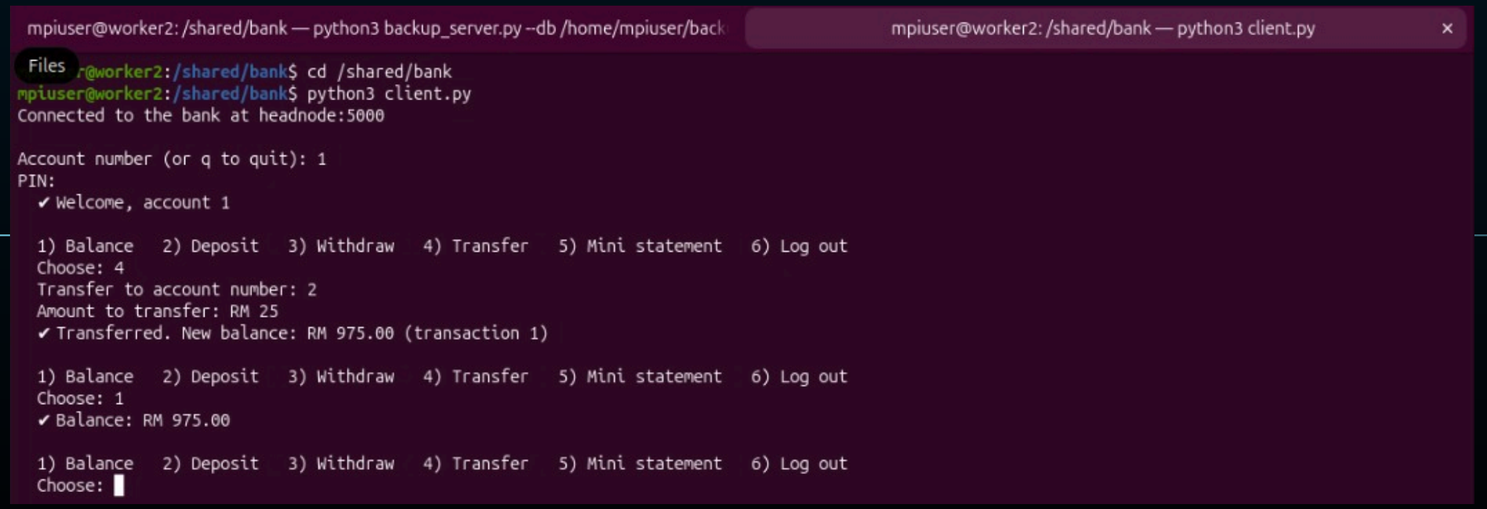
SOFTWARE ARCHITECTURE

- Clients send JSON messages over TCP to port 5000
- Thread pool → lock layer → accounts in memory
- Writer thread saves to SQLite; replicator thread copies to the backup on worker2

CUSTOMERS ON DIFFERENT MACHINES

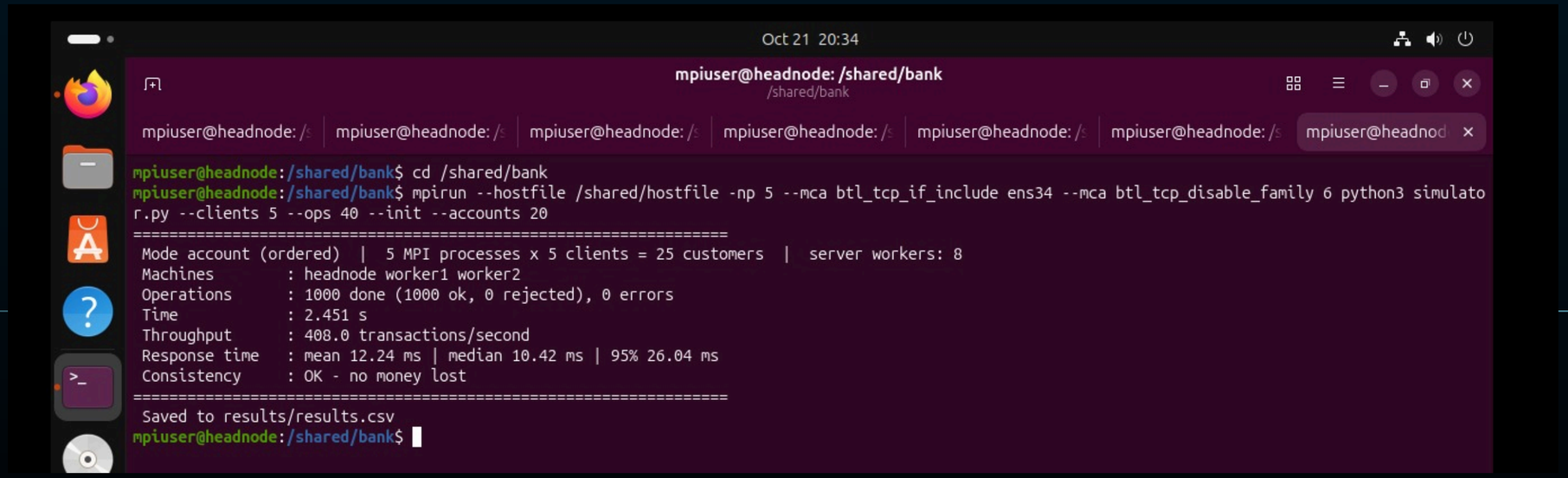


- Customer on worker2 transfers RM 25 from account 1 to account 2
- Customer on worker1 sees account 2 = RM 1,025.00
- Server on headnode, backup on worker2: three machines, one bank



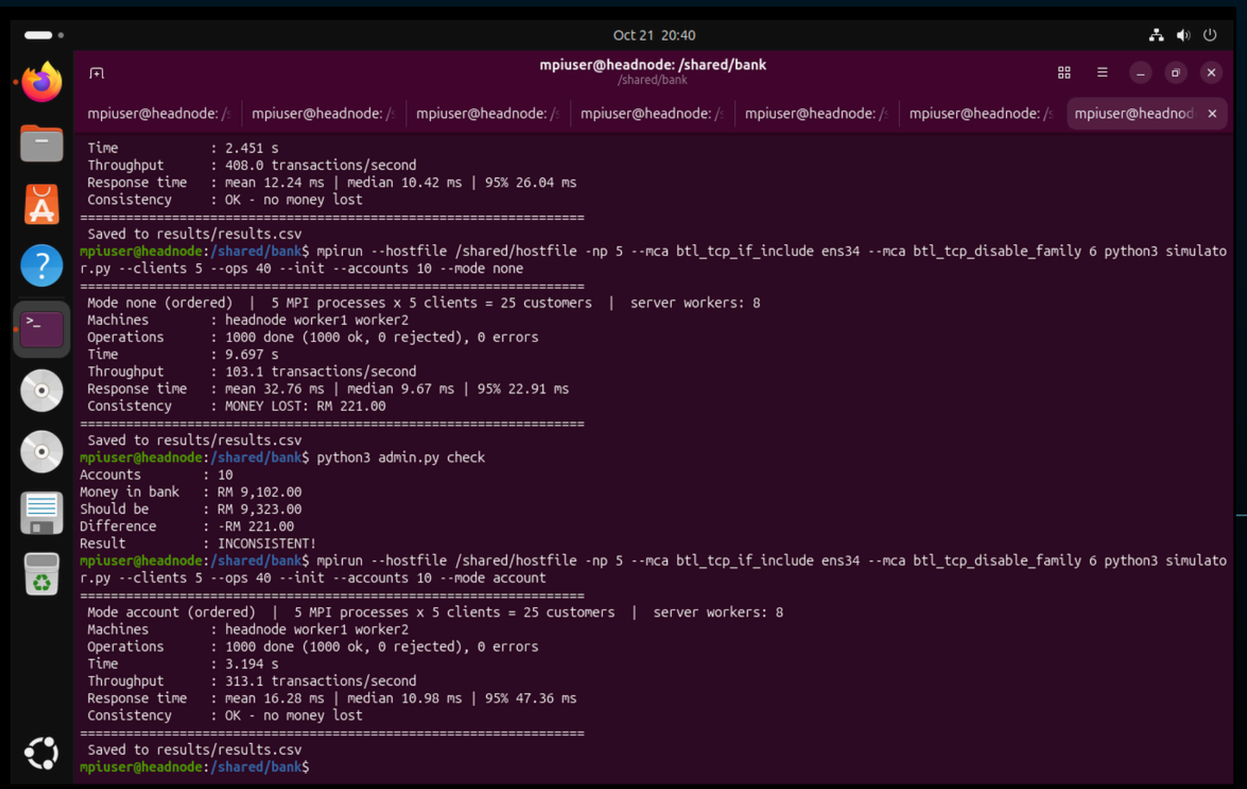
25 CUSTOMERS WITH MPI

- mpirun starts 5 processes on all 3 nodes, each with 5 customer threads
- 25 customers, 1,000 operations
- 408 transactions/second, mean response 12 ms
- No money lost



RACE CONDITION

- Locks off:RM 221 lost, check says INCONSISTENT!
- Locks on: RM 0.00 difference, every time
- Two threads read the same balance; one update overwrites the other
- No-lock runs were wrong every time, by up to RM 601

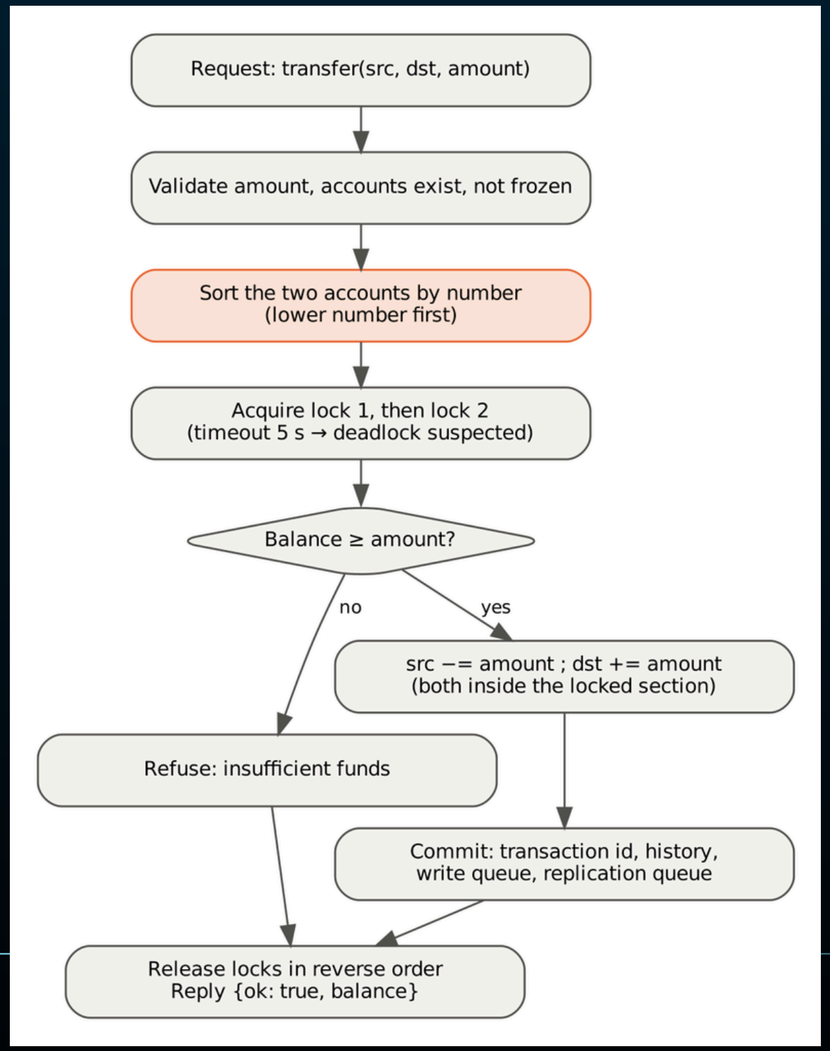


DEADLOCK AND ORDERED LOCKING

	Naive order	Ordered (lower number first)
Deadlocks	**12**	**0**
Transfers done	8 of 20	20 of 20
Time	20.03 s	**0.063 s**

- Ordered locking removes circular wait, so deadlock cannot happen

```
mpiuser@headnode:/shared/bank$ mpirun -np 1 python3 simulator.py --clients 4 --ops 5 --init --pattern pingpong --mode account --lock-order naive
============================================================
 Mode account (naive)  |  1 MPI processes x 4 clients = 4 customers  |  server workers: 8
 Machines        : headnode
 Operations      : 8 done (8 ok, 0 rejected), 12 errors
 Time            : 20.032 s
 Throughput      : 0.4 transactions/second
 Response time   : mean 4004.28 ms | median 5001.47 ms | 95% 5011.32 ms
 DEADLOCK timeouts: 12
 Consistency     : OK - no money lost
============================================================
 Saved to results/results.csv
mpiuser@headnode:/shared/bank$ mpirun -np 1 python3 simulator.py --clients 4 --ops 5 --init --pattern pingpong --mode account --lock-order ordered
============================================================
 Mode account (ordered)  |  1 MPI processes x 4 clients = 4 customers  |  server workers: 8
 Machines        : headnode
 Operations      : 20 done (20 ok, 0 rejected), 0 errors
 Time            : 0.063 s
 Throughput      : 317.5 transactions/second
 Response time   : mean 10.76 ms | median 10.88 ms | 95% 14.1 ms
 Consistency     : OK - no money lost
============================================================
 Saved to results/results.csv
mpiuser@headnode:/shared/bank$ python3 admin.py mode account --lock-order ordered
Lock mode is now account (lock_order=ordered, delay=0.002s)
mpiuser@headnode:/shared/bank$
```



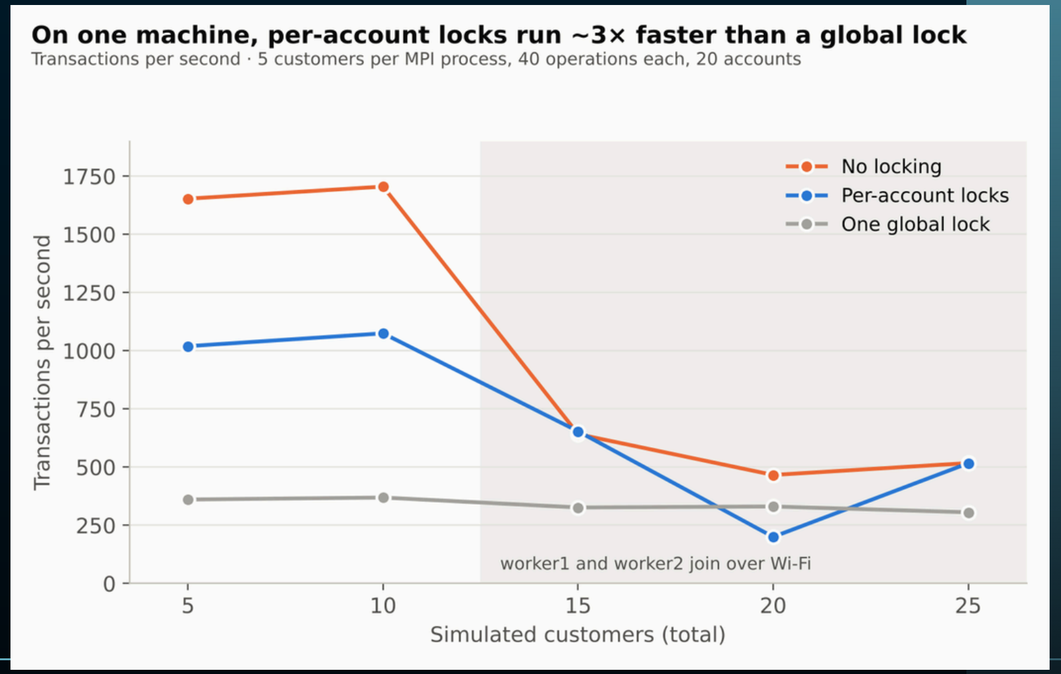
PERFORMANCE AND TESTING

- Per-account locks **2.9× faster** than one global lock (1,074.6 vs 369.1 tx/s)
- Global lock: one transaction at a time, stuck at about 300–370 tx/s
- Across machines, the **Wi-Fi network** became the limit
- **14 of 14** automatic tests passed

```
test_naive_locking_can_deadlock (__main__.Concurrency.test_naive_locking_can_deadlock)
With 'naive' order the opposite transfers block each other (detected by timeout). ... ok
test_no_locking_loses_money (__main__.Concurrency.test_no_locking_loses_money)
Demonstrates the race condition: without locks the totals go wrong. ... ok
test_ordered_locking_never_deadlocks (__main__.Concurrency.test_ordered_locking_never_deadlocks)
Opposite-direction transfers between the same two accounts must all finish. ... ok
test_per_account_locks_keep_money_correct (__main__.Concurrency.test_per_account_locks_keep_money_correct) ... ok

----------------------------------------------------------------------
Ran 14 tests in 4.087s

OK
mpiuser@headnode:/shared/bank$
```



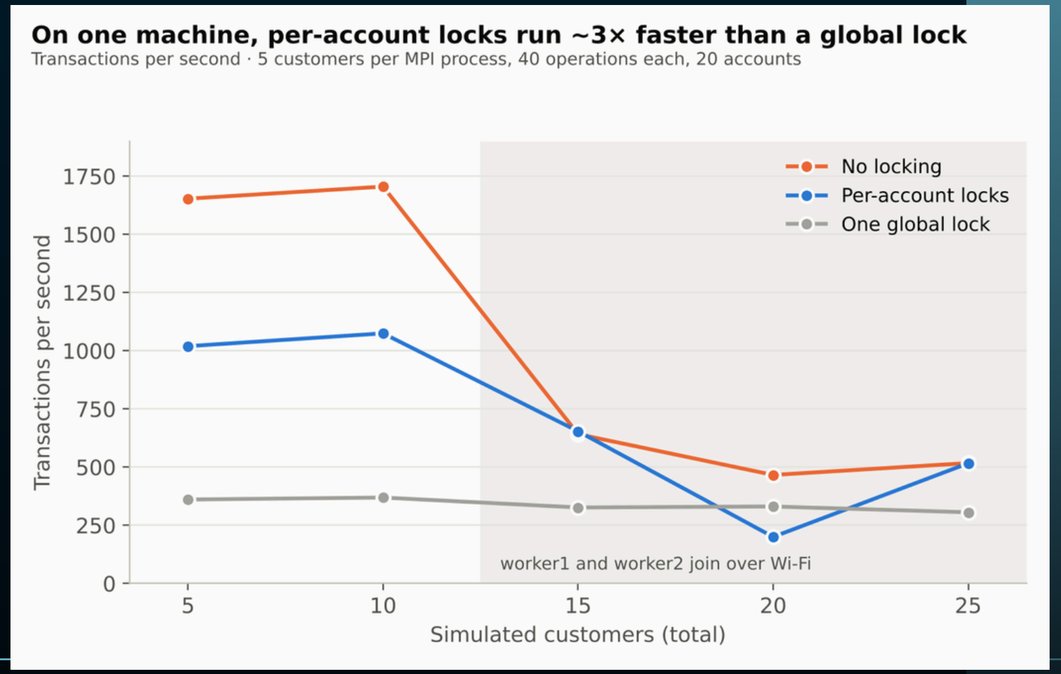
PERFORMANCE AND TESTING

- Per-account locks **2.9× faster** than one global lock (1,074.6 vs 369.1 tx/s)
- Global lock: one transaction at a time, stuck at about 300–370 tx/s
- Across machines, the **Wi-Fi network** became the limit
- **14 of 14** automatic tests passed

```
test_naive_locking_can_deadlock (__main__.Concurrency.test_naive_locking_can_deadlock)
With 'naive' order the opposite transfers block each other (detected by timeout). ... ok
test_no_locking_loses_money (__main__.Concurrency.test_no_locking_loses_money)
Demonstrates the race condition: without locks the totals go wrong. ... ok
test_ordered_locking_never_deadlocks (__main__.Concurrency.test_ordered_locking_never_deadlocks)
Opposite-direction transfers between the same two accounts must all finish. ... ok
test_per_account_locks_keep_money_correct (__main__.Concurrency.test_per_account_locks_keep_money_correct) ... ok

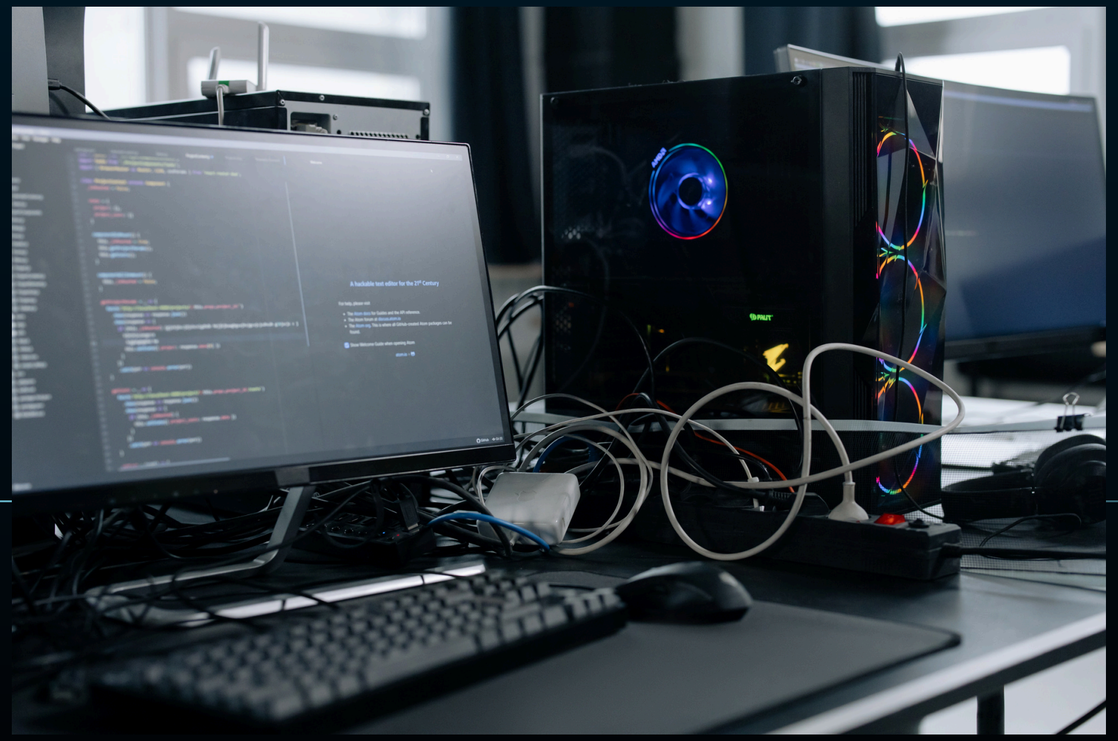
----------------------------------------------------------------------
Ran 14 tests in 4.087s

OK
mpiuser@headnode:/shared/bank$
```



CONCLUSION

- Built a real 3-node cluster and a distributed bank on it
- Locks keep money correct; ordered locking prevents deadlock
- Per-account locks are fastest while staying correct
- Future: wired network, repeated runs, more thread-pool sizes



THANK YOU